121. Best Time to Buy and Sell Stock

LeetCode · Easy · Array · Two Pointers · Greedy
Problem Link: 121. Best Time to Buy and Sell Stock

Problem Statement

You are given an array prices where prices[i] represents the price of a stock on the *i*th day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Examples

Example 1

- **Input:** prices = [7,1,5,3,6,4]
- **Output:** 5
- **Explanation:** Buy on day 2 at price 1 and sell on day 5 at price 6. The profit is 6 - 1 = 5.

Note: Buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.

Example 2

- **Input:** prices = [7,6,4,3,1]
- **Output:** 0
- **Explanation:** No transaction is made because the stock price keeps decreasing. The maximum profit is 0.

Constraints

- 1 <= prices.length <= 10^5
- 0 <= prices[i] <= 10^4

Understanding the Input

1. Understanding the Input

I have given an array named prices, where each element represents the stock price on a particular day. The array contains the prices of a single stock over *n* days, where n = prices.size().

2. Understanding the Indices

Since arrays use zero-based indexing, the first element is at index 0, and the last element is at index n - 1.

Property	Explanation
Array length	n = prices.size()
First index	0
Last index	n - 1
Valid index range	0 to n - 1

3. Understanding the Buying and Selling Rules

I can buy the stock on one day and sell it on a different day in the future. This means the buying day must always come before the selling day.

- I must buy the stock before selling it.
- I can perform only one buy-and-sell transaction.
- I cannot buy and sell on the same day.
- I cannot buy on a later day and sell on an earlier day.

4. Understanding the Main Objective

My goal is to find the maximum profit possible from a single buy-and-sell transaction.

The profit is calculated by subtracting the buying price from the selling price.

Profit = Selling Price − Buying Price

I need to choose the buying and selling days such that the profit is as large as possible. If no transaction can produce a positive profit, I must return 0.

5. Understanding the Constraints

Constraint	Meaning
1 <= prices.length <= 100000	The array contains at least one day and at most 10^5 days.
0 <= prices[i] <= 10000	Each stock price can range from 0 to 10000.

Since the array can contain up to 10^5 elements, my approach should be efficient enough to handle a large input.

6. Visualizing the Example

For prices = [7, 1, 5, 3, 6, 4], the stock prices over the six days are:

Day (Index)	Stock Price
0	7
1	1
2	5
3	3
4	6
5	4

The objective is to identify the best buying day and a subsequent selling day that produce the maximum profit.

Approach 1 — Brute Force

Status: Implemented · **LeetCode:** Easy

1. Initial Intuition

My first thought is to check every possible combination of buying and selling days to find the maximum profit.

Since I am allowed to buy the stock on one day and sell it only on a later day, I need to consider every valid pair of days.

For example, if I buy on day i, I can sell on any day j where j > i. For each such pair, I calculate the profit and compare it with the maximum profit I have found so far.

I do not need to perform a transaction if it results in a loss or zero profit because the answer should never be less than 0.

The main idea is to try every valid buying-and-selling combination and keep track of the maximum profit.

2. My Approach

Step 1: Initialize the variables

- n = prices.size() stores the total number of days.
- profit = 0 stores the maximum profit found so far.
- I initialize profit to 0 because, if no profitable transaction exists, I must return 0.

Step 2: Select a buying day

I use an outer loop with index i to select each possible buying day. The loop runs from i = 0 to i = n - 2 because at least one later day must exist to sell the stock.

Step 3: Select a selling day

For every buying day i, I use an inner loop with index j to examine every subsequent day. The inner loop starts at j = i + 1, ensuring that the selling day always comes after the buying day.

Step 4: Calculate the current profit

For every valid pair of days, I calculate the difference between the selling price and the buying price.

currentProfit = prices[j] - prices[i]

- If the selling price is higher than the buying price, the profit is positive.
- If both prices are equal, the profit is 0.
- If the selling price is lower than the buying price, the result is negative.

Step 5: Update the maximum profit

I compare currentProfit with profit. If currentProfit > profit, I update profit with the new value. Otherwise, I keep the previous maximum profit.

Step 6: Return the answer

After examining every valid pair of days, I return profit, which contains the maximum possible profit.

3. C++ Implementation

```cpp
// Problem: 121. Best Time to Buy and Sell Stock
// Approach 1: Brute Force
// Time Complexity: O(n^2)
// Auxiliary Space: O(1)

#include <iostream>
#include <vector>
using namespace std;

// ==================== LEETCODE SOLUTION ====================

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        int profit = 0;

        // Try every buying day
        for (int i = 0; i < n - 1; i++) {

            // Try every selling day after buying
            for (int j = i + 1; j < n; j++) {
                int currentProfit = prices[j] - prices[i];

                // Update maximum profit
                if (currentProfit > profit) {
                    profit = currentProfit;
                }
            }
        }

        return profit;
    }
};

// ==================== LOCAL TESTING ====================

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

3.1 Understanding the Important Code

Code	Explanation
int n = prices.size();	Stores the total number of stock prices.
int profit = 0;	Initializes the maximum profit to 0.
for (int i = 0; i < n - 1; i++)	Selects each possible buying day except the last day.
for (int j = i + 1; j < n; j++)	Checks every selling day after the selected buying day.
prices[j] - prices[i]	Calculates the profit for the current transaction.
if (currentProfit > profit)	Checks whether the current transaction produces a better profit.
profit = currentProfit;	Updates the maximum profit when a better transaction is found.
return profit;	Returns the maximum profit after checking all valid pairs.

Why do we start j from i + 1?
Because the buying day must come before the selling day. Starting j at i + 1 ensures that every transaction follows this rule without needing an additional condition.

Why do we initialize profit = 0?
Because the problem allows us to make no transaction. If every possible transaction results in a loss, the answer must remain 0.

4. Dry Run — Step-by-Step Execution

Input: prices = [7, 1, 5, 3, 6, 4]

Initially:

Variable	Value	Meaning
n	6	There are six days.
profit	0	No profitable transaction has been found yet.

4.1 Understanding the Input

Day	Index	Stock Price
Day 1	0	7
Day 2	1	1
Day 3	2	5
Day 4	3	3
Day 5	4	6
Day 6	5	4

The outer loop selects the buying day, while the inner loop checks every possible selling day after it.

4.2 Buying on Day 1 — Price 7

The outer loop sets i = 0, so the buying price is prices[0] = 7. The inner loop checks every later day.

Buying Price	Selling Price	Calculation	Current Profit	Maximum Profit
7	1	1 - 7	-6	0
7	5	5 - 7	-2	0
7	3	3 - 7	-4	0
7	6	6 - 7	-1	0
7	4	4 - 7	-3	0

Observation: Every transaction results in a loss. Since none of the current profits is greater than 0, the maximum profit remains unchanged.

After completing this iteration: profit = 0

4.3 Buying on Day 2 — Price 1

The outer loop moves to i = 1, so the buying price is prices[1] = 1. The inner loop now checks the remaining days.

Buying Price	Selling Price	Calculation	Current Profit	Maximum Profit
1	5	5 - 1	4	4
1	3	3 - 1	2	4
1	6	6 - 1	5	5
1	4	4 - 1	3	5

Let's understand how profit changes:

1. Sell at 5: the profit is 4. Since 4 > 0, update profit = 4.
2. Sell at 3: the profit is 2. Since 2 is less than 4, keep profit = 4.
3. Sell at 6: the profit is 5. Since 5 > 4, update profit = 5.
4. Sell at 4: the profit is 3. Since 3 is less than 5, keep profit = 5.

Observation: Buying at 1 and selling at 6 produces a profit of 5, which becomes the highest profit found so far.

After completing this iteration: profit = 5

4.4 Buying on Day 3 — Price 5

The outer loop moves to i = 2, so the buying price is 5.

Buying Price	Selling Price	Calculation	Current Profit	Maximum Profit
5	3	3 - 5	-2	5
5	6	6 - 5	1	5
5	4	4 - 5	-1	5

Observation: The best profit from buying at 5 is 1. Since the maximum profit is already 5, no update is needed.

After completing this iteration: profit = 5

4.5 Buying on Day 4 — Price 3

The outer loop moves to i = 3, so the buying price is 3.

Buying Price	Selling Price	Calculation	Current Profit	Maximum Profit
3	6	6 - 3	3	5
3	4	4 - 3	1	5

Observation: The best profit from buying at 3 is 3. This is still less than the maximum profit of 5.

After completing this iteration: profit = 5

4.6 Buying on Day 5 — Price 6

The outer loop moves to i = 4, so the buying price is 6.

Buying Price	Selling Price	Calculation	Current Profit	Maximum Profit
6	4	4 - 6	-2	5

Observation: Selling at 4 results in a loss. The maximum profit remains 5.

After completing this iteration: profit = 5

4.7 Buying on Day 6 — Price 4

The last day cannot be selected as a buying day because there is no later day available for selling. The outer loop condition is i < n - 1, so when i = 5, the condition becomes 5 < 5, which is false. The loop terminates.

No further transactions are checked.

5. Final Dry Run Summary

Buying Day	Buying Price	Best Selling Price After Buying	Best Profit for That Day	Overall Maximum
Day 1	7	6	-1	0
Day 2	1	6	5	5
Day 3	5	6	1	5
Day 4	3	6	3	5
Day 5	6	4	-2	5

The final answer is:

Maximum Profit: 5

The best transaction is:

- **Buying day:** Day 2, at price 1.
- **Selling day:** Day 5, at price 6.
- **Maximum profit:** 6 - 1 = 5.

6. Complexity Analysis

6.1 Time Complexity — $O(n^2)$

The algorithm uses two nested loops.

- The outer loop selects each possible buying day.
- For each buying day, the inner loop examines every subsequent selling day.

For an array of length n, the number of valid pairs is:

(n - 1) + (n - 2) + ... + 1

Using the sum of the first n - 1 positive integers:

n(n - 1) / 2

Expanding the expression gives:

$(n^2 - n) / 2$

The dominant term is n^2. We ignore constant factors and lower-order terms when expressing Big-O complexity.

Therefore, the time complexity is **$O(n^2)$**.

Why is this inefficient?

Suppose the array contains 100,000 stock prices. The number of valid buying-and-selling pairs is:

(100000 × 99999) / 2 = 4,999,950,000

That is almost five billion pairs to examine. Consequently, the brute-force approach becomes too slow for large inputs.

6.2 Auxiliary Space Complexity — O(1)

Auxiliary space measures the extra memory used by the algorithm, excluding the input array.

Our algorithm uses only a fixed number of variables:

Variable	Purpose
n	Stores the array length.
profit	Stores the maximum profit.
i	Tracks the buying day.
j	Tracks the selling day.
currentProfit	Stores the profit for the current pair.

The number of variables does not increase as the array grows. We do not create another array, a map, or any other data structure proportional to the input size.

Therefore, the auxiliary space complexity is **O(1)**.

Note that the input vector itself occupies O(n) space, but it is provided to the function and is not additional auxiliary space.

7. What I Learned

- Brute force is a natural starting point because it checks every valid transaction.
- Nested loops allow me to examine every possible buying-and-selling pair.
- Starting the inner loop at i + 1 ensures that buying always happens before selling.
- Initializing the maximum profit to 0 correctly handles cases where no profitable transaction exists.
- The approach takes $O(n^2)$ time because it examines every valid pair.
- The approach uses O(1) auxiliary space because it requires only a fixed number of variables.
- Although the approach is easy to understand and implement, its time complexity makes it inefficient for large inputs.

Approach 2 — Suffix Maximum Using max_element()

Status: Implemented · **Time:** O(n^2) · **Auxiliary Space:** O(1)

1. The New Intuition

In Approach 1, I checked every possible buying day and every possible selling day after it. For each pair, I calculated the profit separately.

Now, I want to make the logic more direct.

For any chosen buying day, the best selling day is the day with the maximum stock price among all the future days.

For example, consider:

prices = {7, 1, 5, 3, 6, 4}

Suppose I choose to buy at price 1. The future prices are {5, 3, 6, 4}.

Property	Value
Buying price	1
Future prices	5, 3, 6, 4
Maximum future price	6
Maximum profit for this buying day	6 - 1 = 5

This gives me the best possible profit for that particular buying day.

I can repeat the same process for every possible buying day and keep track of the maximum profit.

My key observation

In Approach 1, I calculate the profit for every valid pair. In Approach 2, I find the maximum future selling price first and calculate just one profit for each buying day.

This simplifies the calculation, but I still need to find the maximum future price separately for every buying day.

2. My Approach — Step by Step

Step 1: Initialize the variables

- n = prices.size() stores the total number of days.
- profit = 0 stores the maximum profit found so far.

I initialize profit to 0 because I should never return a negative profit.

Step 2: Select a buying day

I use an outer loop to select each possible buying day, from index 0 to n - 2. I stop at n - 2 because I need at least one future day to sell the stock.

Step 3: Find the maximum future price

For every buying day i, I use max_element() to find the highest stock price in the range after that day.

The range is:

prices.begin() + i + 1 to prices.end()

This ensures that the selling day always comes after the buying day.

Step 4: Calculate the best profit for that buying day

Once I find the maximum future price, I subtract the buying price from it.

currentProfit = maxPrice - prices[i]

This gives me the maximum possible profit for the selected buying day.

Step 5: Update the maximum profit

I compare currentProfit with the current profit. If currentProfit > profit, I update profit. Otherwise, I retain the previous maximum.

Step 6: Return the answer

After checking every possible buying day, I return profit.

3. Understanding max_element()

The C++ Standard Template Library (STL) provides max_element() in the <algorithm> header. It returns an iterator pointing to the largest element in the specified range.

Syntax

```
max_element(first, last);
```

Here, first is the beginning of the range, and last is the exclusive end boundary.

Understanding our expression

```
int maxPrice = *max_element(
    prices.begin() + i + 1,
    prices.end()
);
```

Expression	Meaning
prices.begin()	Returns an iterator to the first element of the vector.
prices.begin() + i + 1	Points to the element immediately after the buying day.
prices.end()	Points to the position just past the last element. It is not an actual element.
max_element(...)	Finds the maximum element within the specified range and returns its iterator.
*max_element(...)	Dereferences the returned iterator to obtain the actual stock price.
int maxPrice = ...	Stores the maximum future stock price in maxPrice.

Example of the range

Suppose i = 1 and:

prices = {7, 1, 5, 3, 6, 4}

Why do we start at i + 1 instead of i?

Because we cannot sell the stock on the same day we buy it. Starting at i + 1 guarantees that every candidate selling day is in the future.

Important: max_element() finds the maximum price, but it does not calculate the profit. We calculate the profit separately by subtracting the buying price.

4. C++ Implementation

```
// Problem: 121. Best Time to Buy and Sell Stock
// Approach 2: Suffix Maximum using max_element()
// Time Complexity: O(n^2)
// Auxiliary Space: O(1)

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// ==================== LEETCODE SOLUTION ====================

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();
        int profit = 0;

        for (int i = 0; i < n - 1; i++) {

            // Find maximum price after buying day i
            int maxPrice = *max_element(
                prices.begin() + i + 1,
                prices.end()
            );

            int currentProfit = maxPrice - prices[i];

            // Update maximum profit
            if (currentProfit > profit) {
                profit = currentProfit;
            }
        }

        return profit;
    }
};

// ==================== LOCAL TESTING ====================

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

4.1 Important Code Details

Code	Purpose
int n = prices.size();	Stores the number of days.
int profit = 0;	Tracks the maximum profit found so far.
i < n - 1	Ensures that the buying day has at least one future day.
max_element(...)	Finds the maximum selling price among the future days.
maxPrice - prices[i]	Calculates the best profit for the current buying day.

Code	Purpose
if (currentProfit > profit)	Updates the answer only when a larger profit is found.
return profit;	Returns the maximum profit.

5. Complete Dry Run

Input: prices = {7, 1, 5, 3, 6, 4}

Initially:

Variable	Value
n	6
profit	0

For every buying day, the algorithm scans the future portion of the array to find the maximum price.

Iteration 1 — Buy at price 7

Buying index: i = 0
Future prices: {1, 5, 3, 6, 4}

Step	Value
Buying price	7
Maximum future price	6
Current profit	6 - 7 = -1
Previous maximum profit	0
Updated maximum profit	0

Since -1 is less than 0, the maximum profit remains unchanged.

Iteration 2 — Buy at price 1

Buying index: i = 1
Future prices: {5, 3, 6, 4}

Step	Value
Buying price	1
Maximum future price	6
Current profit	6 - 1 = 5
Previous maximum profit	0
Updated maximum profit	5

Since 5 is greater than 0, we update profit = 5.

Iteration 3 — Buy at price 5

Buying index: i = 2
Future prices: {3, 6, 4}

Step	Value
Buying price	5
Maximum future price	6
Current profit	6 - 5 = 1
Previous maximum profit	5
Updated maximum profit	5

Since 1 is less than 5, the maximum profit remains 5.

Iteration 4 — Buy at price 3

Buying index: i = 3
Future prices: {6, 4}

Step	Value
Buying price	3
Maximum future price	6
Current profit	6 - 3 = 3
Previous maximum profit	5
Updated maximum profit	5

Since 3 is less than 5, the maximum profit remains 5.

Iteration 5 — Buy at price 6

Buying index: i = 4
Future prices: {4}

Step	Value
Buying price	6
Maximum future price	4
Current profit	4 - 6 = -2
Previous maximum profit	5
Updated maximum profit	5

Since -2 is less than 5, the maximum profit remains 5.

Iteration 6 — Index 5

The last day cannot be a buying day because there is no future day available to sell the stock. The outer loop terminates when i = n - 1.

Final Dry Run Summary

Buying Index	Buying Price	Future Prices	Maximum Future Price	Current Profit	Maximum Profit
0	7	{1, 5, 3, 6, 4}	6	-1	0

Buying Index	Buying Price	Future Prices	Maximum Future Price	Current Profit	Maximum Profit
1	1	{5, 3, 6, 4}	6	5	5
2	5	{3, 6, 4}	6	1	5
3	3	{6, 4}	6	3	5
4	6	{4}	4	-2	5

Final output: Maximum Profit: 5

The maximum profit is achieved by buying at price 1 and selling later at price 6.

6. Complexity Analysis

6.1 Time Complexity — $O(n^2)$

The outer loop runs approximately n - 1 times. During each iteration, max_element() scans the future portion of the array to find the largest price.

Buying index	Number of elements scanned
0	n - 1
1	n - 2
2	n - 3
...	...
n - 2	1

The total number of elements inspected is:

(n - 1) + (n - 2) + ... + 1

Using the sum formula:

n(n - 1) / 2

Expanding:

$(n^2 - n) / 2$

The dominant term is n^2, so the time complexity is **$O(n^2)$**.

Although max_element() makes the code more concise, it still scans the future elements repeatedly. We have simplified the logic, but we have not improved the asymptotic time complexity compared with brute force.

6.2 Auxiliary Space Complexity — O(1)

The algorithm uses a fixed number of variables: n, profit, i, maxPrice, and currentProfit.

max_element() returns an iterator and does not create another array containing the future prices.

Therefore, the auxiliary space complexity is **O(1)**.

7. What Improved, and What Did Not?

Aspect	Approach 1: Brute Force	Approach 2: max_element()
Main idea	Check every buy-and-sell pair	Find the best future selling price for each buying day
Profit calculations	Multiple per buying day	One per buying day
Finding the selling price	Examine each pair directly	Find the maximum future price first
Time complexity	$O(n^2)$	$O(n^2)$
Auxiliary space	O(1)	O(1)
Main limitation	Repeats pairwise profit calculations	Repeatedly scans overlapping future ranges

8. What I Learned

- For a fixed buying day, the maximum future selling price gives the best possible profit for that day.
- max_element() helps find the maximum element in a range without writing another loop.
- Starting the range at i + 1 ensures that the selling day is later than the buying day.
- The dereference operator * lets me obtain the value from the iterator returned by max_element().
- This approach improves the clarity of the logic, but its time complexity remains $O(n^2)$.
- The future ranges overlap, so many of the same prices are scanned repeatedly.

My next question

What if I calculate the maximum future price for every index once and store those values in a separate array?

That idea leads to the next approach: **Suffix Maximum Array**, where I can avoid repeatedly scanning the same future prices.

Approach 3 — Suffix Maximum Array

1. Initial Intuition

In Approach 2, we used max_element() to find the highest stock price after each possible buying day. The problem was that max_element() scanned the remaining array repeatedly, resulting in $O(n^2)$ time complexity.

Instead of repeatedly searching for the maximum future price, we can calculate and store the maximum future price for every index in advance. This creates a suffix maximum array.

For every index i, suffixMax[i] stores the maximum price from index i to the last index.

For example:

Index	0	1	2	3	4	5
prices	7	1	5	3	6	4
suffixMax	7	6	6	6	6	4

My observation: Once the suffix maximum array is built, I can find the best future selling price in O(1) time for each buying day.

2. How to Build the Suffix Maximum Array

We build the array from right to left because each position depends on the maximum value already calculated for the position to its right.

Step 1: Initialize the last element

The last element has no elements after it, so its suffix maximum is the element itself.

suffixMax[n - 1] = prices[n - 1]

For our input, suffixMax[5] = 4.

Step 2: Move from right to left

At every index i, the maximum price from that index onward must be one of these two values:

1. The current price: prices[i]
2. The maximum price from the next index onward: suffixMax[i + 1]

Therefore:

```
suffixMax[i] = max(prices[i], suffixMax[i + 1]);
```

Building it step by step

Index i	Current price	Suffix maximum to the right	Calculation	suffixMax[i]
5	4	—	Initialize last element	4
4	6	4	max(6, 4)	6
3	3	6	max(3, 6)	6
2	5	6	max(5, 6)	6
1	1	6	max(1, 6)	6

Index i	Current price	Suffix maximum to the right	Calculation	suffixMax[i]
0	7	6	max(7, 6)	7

The completed array is:

suffixMax = [7, 6, 6, 6, 6, 4]

3. How to Calculate the Maximum Profit

For every buying day i, we use the maximum price available after that day.

Current Profit = Best Future Selling Price − Buying Price

In our code:

```
int currentProfit = suffixMax[i + 1] - prices[i];
```

Why i + 1? The selling day must be strictly after the buying day. Using suffixMax[i] could select the current day's price, which would allow buying and selling on the same day. Using suffixMax[i + 1] prevents that.

4. Complete C++ Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Problem: 121. Best Time to Buy and Sell Stock
// Approach 3: Suffix Maximum Array
// Time Complexity: O(n)
// Auxiliary Space: O(n)

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();

        vector<int> suffixMax(n);
        suffixMax[n - 1] = prices[n - 1];

        // Build suffix maximum from right to left
        for (int i = n - 2; i >= 0; i--) {
            suffixMax[i] = max(prices[i], suffixMax[i + 1]);
        }

        int profit = 0;

        // Find profit using the best future selling price
        for (int i = 0; i < n - 1; i++) {
            int currentProfit = suffixMax[i + 1] - prices[i];

            if (currentProfit > profit) {
                profit = currentProfit;
            }
        }

        return profit;
    }
};

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

5. Detailed Dry Run

Input: prices = [7, 1, 5, 3, 6, 4]

Phase 1: Build the suffix maximum array

We have already calculated:

suffixMax = [7, 6, 6, 6, 6, 4]

Phase 2: Calculate profit for each buying day

Initially, profit = 0.

Buying index i	Buying price	suffixMax[i + 1]	Current profit	Best profit so far
0	7	6	6 - 7 = -1	0
1	1	6	6 - 1 = 5	5
2	5	6	6 - 5 = 1	5
3	3	6	6 - 3 = 3	5
4	6	4	4 - 6 = -2	5

Index 5 is not considered as a buying day because there is no later day available for selling.

Final output: Maximum Profit: 5

The best transaction is to buy at price 1 on index 1 and sell at price 6 on index 4.

6. Complexity Analysis

Time complexity: O(n)

There are two separate loops:

- Building suffixMax processes the array once: O(n).
- Calculating the profit processes the array once more: O(n).

Therefore, the total is O(n) + O(n) = O(n).

Auxiliary space: O(n)

The suffixMax vector stores one value for every input index. For n prices, it requires n additional elements.

7. What I Learned

- Repeatedly scanning future prices can be avoided by precomputing their maximum values.
- A suffix maximum stores the maximum value from the current index to the end.
- Suffix information is built from right to left.
- Precomputation improves the time complexity of this solution from $O(n^2)$ to O(n), at the cost of O(n) extra space.

My next question

Do I really need to store every future maximum? Could I solve the problem by remembering the lowest price from previous days instead?

That leads to **Approach 4**.

Approach 4 — Prefix Minimum Array

1. Initial Intuition

In Approach 3, we looked toward the future to find the highest selling price available after each buying day. Now, we can look at the problem from the opposite direction.

Instead of asking: **What is the highest price after my buying day?**

We ask: **What is the lowest price before my selling day?**

For any selling day, the best possible profit comes from buying at the lowest price on an earlier day.

To avoid repeatedly searching all previous prices, we can store the minimum price seen so far in a prefix minimum array.

For every index i, prefixMin[i] stores the minimum price from index 0 through index i.

For our input:

Index	0	1	2	3	4	5
prices	7	1	5	3	6	4
prefixMin	7	1	1	1	1	1

My observation: Instead of storing the best future selling price, I can store the best earlier buying price.

2. How to Build the Prefix Minimum Array

We build the array from left to right because each position depends on the minimum value calculated for the previous position.

Step 1: Initialize the first element

At index 0, only one price has been seen, so the minimum is the price itself.

```
prefixMin[0] = prices[0];
```

For our input, prefixMin[0] = 7.

Step 2: Move from left to right

At each index i, the minimum price seen so far must be one of these two values:

1. The current price: prices[i]
2. The minimum price from the previous prefix: prefixMin[i - 1]

Therefore:

```
prefixMin[i] = min(prefixMin[i - 1], prices[i]);
```

Building it step by step

Index i	Current price	Previous prefix minimum	Calculation	prefixMin[i]
0	7	—	Initialize first	7

Index i	Current price	Previous prefix minimum	Calculation	prefixMin[i]
			element	
1	1	7	min(7, 1)	1
2	5	1	min(1, 5)	1
3	3	1	min(1, 3)	1
4	6	1	min(1, 6)	1
5	4	1	min(1, 4)	1

The completed array is:

prefixMin = [7, 1, 1, 1, 1, 1]

3. How to Calculate the Maximum Profit

Now, we treat each day as a possible selling day. For a selling day i, we need the minimum buying price among all days before it.

Current Profit = Selling Price − Lowest Earlier Buying Price

In our code:

```
int currentProfit = prices[i] - prefixMin[i - 1];
```

Why i - 1? Because buying must happen before selling. prefixMin[i - 1] contains the minimum price from index 0 to index i - 1, so it excludes the current selling day.

The profit loop starts at i = 1 because index 0 has no earlier day on which to buy.

4. Complete C++ Code

```cpp
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Problem: 121. Best Time to Buy and Sell Stock
// Approach 4: Prefix Minimum Array
// Time Complexity: O(n)
// Auxiliary Space: O(n)

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();

        vector<int> prefixMin(n);
        prefixMin[0] = prices[0];

        // Build prefix minimum from left to right
        for (int i = 1; i < n; i++) {
            prefixMin[i] = min(prefixMin[i - 1], prices[i]);
        }

        int profit = 0;

        // Find profit using the best earlier buying price
        for (int i = 1; i < n; i++) {
            int currentProfit = prices[i] - prefixMin[i - 1];

            if (currentProfit > profit) {
                profit = currentProfit;
            }
        }

        return profit;
    }
};

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

5. Detailed Dry Run

Input: prices = [7, 1, 5, 3, 6, 4]

Phase 1: Build the prefix minimum array

The result is:

prefixMin = [7, 1, 1, 1, 1, 1]

Phase 2: Calculate profit for each selling day

Initially, profit = 0.

Selling index i	Selling price	prefixMin[i - 1]	Current profit	Best profit so far
1	1	7	1 - 7 = -6	0
2	5	1	5 - 1 = 4	4
3	3	1	3 - 1 = 2	4

Selling index i	Selling price	prefixMin[i - 1]	Current profit	Best profit so far
4	6	1	6 - 1 = 5	5
5	4	1	4 - 1 = 3	5

Notice that the best profit becomes 5 when the selling price reaches 6 at index 4. The algorithm does not update the maximum profit when the current profit is lower than the best profit already found.

Final output: Maximum Profit: 5

The best transaction is to buy at price 1 on index 1 and sell at price 6 on index 4.

6. Complexity Analysis

Time complexity: O(n)

There are two passes through the array:

- Building prefixMin: O(n).
- Calculating the maximum profit: O(n).

Total: O(n) + O(n) = O(n).

Auxiliary space: O(n)

The prefixMin vector stores the minimum price for every index, requiring n additional elements.

7. Connection Between Approach 3 and Approach 4

Both approaches solve the same problem in linear time, but they precompute different information.

Basis	Approach 3: Suffix Maximum	Approach 4: Prefix Minimum
Main idea	Find the best future selling price	Find the best earlier buying price
Looks toward	Future days	Previous days
Array stores	Maximum price from index i to the end	Minimum price from index 0 to i
Construction direction	Right to left	Left to right
Main function	max()	min()
Profit calculation	suffixMax[i + 1] - prices[i]	prices[i] - prefixMin[i - 1]
Time complexity	O(n)	O(n)
Auxiliary space	O(n)	O(n)

Why do both approaches work?

A profitable transaction needs two things:

- A buying price that is as low as possible.
- A selling price that is as high as possible, while occurring after the buying day.

The suffix maximum approach fixes a buying day and searches for its best future selling price. The prefix minimum approach fixes a selling day and searches for its best earlier buying price.

Both examine every valid opportunity through precomputed information, so neither needs to repeatedly scan the array.

8. What I Learned

- A prefix minimum stores the lowest value encountered from the beginning through the current index.
- Prefix information is built from left to right.
- Using prefixMin[i - 1] ensures the buying day occurs before the selling day.
- Precomputing an array gives us O(n) time, but requires O(n) additional space.
- Both prefix minimum and suffix maximum arrays solve the problem, but each stores information from a different direction.

9. What Can I Improve Now?

Look at the prefix minimum array again:

prefixMin = [7, 1, 1, 1, 1, 1]

When calculating profit for a selling day, do I really need the entire array? I only need the lowest price seen before the current day.

I can maintain that information in a single variable called minPrice, updating it as I move through the array. That removes the extra array while keeping the time complexity at O(n).

This leads to **Approach 5 — Greedy: Track Minimum Price**. It achieves O(n) time and O(1) auxiliary space, making it the most space-efficient of these approaches.

Approach 5 — Greedy: Track Minimum Price

Status: Implemented · **Optimal approach** · **Time:** O(n) · **Auxiliary Space:** O(1)

1. Initial Intuition

In the previous approaches, I explored different ways to find the maximum profit:

- Brute force: Check every valid buying-and-selling pair.
- Using max_element(): Find the maximum future selling price for every buying day.
- Prefix minimum array: Store the minimum price encountered up to each index.

Now, I want to solve the problem without repeatedly checking pairs or storing extra arrays.

My key observation is that to calculate the best profit for today's selling price, I only need to know the lowest stock price encountered before today.

For example, consider:

prices = {7, 1, 5, 3, 6, 4}

Suppose today’s price is 6. I do not need to check every earlier price individually. If I already know that the minimum earlier price was 1, I can calculate the profit directly:

currentProfit = 6 - 1 = 5

This means I can process the array from left to right while maintaining only two important values:

- minPrice: the lowest stock price encountered so far.
- maxProfit: the maximum profit found so far.

Instead of storing the minimum price at every index in a separate array, I update a single variable whenever I encounter a lower price.

2. My Approach — Step by Step

Step 1: Initialize the variables

I initialize:

- minPrice = prices[0] — the first day's price becomes my initial minimum buying price.
- maxProfit = 0 — no positive profit has been found yet.

I start with the first price because the array contains at least one element.

Step 2: Traverse the array from left to right

I start the loop from index 1 because index 0 has already been used to initialize minPrice. For every index i, I treat prices[i] as today's selling price.

Step 3: Calculate today's profit

I calculate the profit using the lowest buying price encountered on earlier days:

currentProfit = prices[i] - minPrice

Because minPrice comes from an earlier index, this calculation respects the rule that buying must happen before selling.

Step 4: Update the maximum profit

I compare currentProfit with maxProfit.

- If currentProfit is greater, I update maxProfit.
- Otherwise, I retain the previous maximum profit.

This ensures that maxProfit always stores the best valid profit found so far.

Step 5: Update the minimum buying price

After evaluating today's selling opportunity, I check whether today's price is lower than minPrice. If it is, I update:

minPrice = prices[i]

This new minimum can be used as the buying price for future selling days.

Step 6: Return the answer

After traversing the entire array, I return maxProfit.

At this point, I have considered every day as a potential selling day and calculated its best possible profit using the lowest earlier buying price.

3. Why Is This a Greedy Approach?

A greedy approach makes a locally best choice at each step to help reach the overall optimal answer.

In this problem, for every selling day, I use the minimum stock price encountered before that day as the best buying price.

Why does this work?

Suppose today's selling price is 6, and the earlier prices are 7, 1, 5, 3.

Earlier Buying Price	Selling Price	Profit
7	6	-1
1	6	5
5	6	1
3	6	3

The lowest earlier price, 1, gives the highest profit of 5.

I do not need to compare today's selling price against all the earlier prices because I already know the minimum earlier price. Therefore, I only need to remember the best buying price seen so far and the maximum profit found so far.

Important: minPrice is not necessarily the final buying price. It is the lowest price encountered so far, and it can change as the algorithm moves through the array.

4. C++ Implementation

```cpp
// Problem: 121. Best Time to Buy and Sell Stock
// Approach 5: Greedy – Track Minimum Price
// Time Complexity: O(n)
// Auxiliary Space: O(1)

#include <iostream>
#include <vector>
using namespace std;

// ==================== LEETCODE SOLUTION ====================

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();

        int minPrice = prices[0];
        int maxProfit = 0;

        // Traverse the prices once
        for (int i = 1; i < n; i++) {

            // Calculate profit using the lowest earlier price
            int currentProfit = prices[i] - minPrice;

            // Update maximum profit
            if (currentProfit > maxProfit) {
                maxProfit = currentProfit;
            }

            // Update minimum buying price
            if (prices[i] < minPrice) {
                minPrice = prices[i];
            }
        }

        return maxProfit;
    }
};

// ==================== LOCAL TESTING ====================

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

5. Understanding the Important Code

Code	Explanation
int n = prices.size();	Stores the total number of days.
int minPrice = prices[0];	Initializes the minimum price with the first day's price.
int maxProfit = 0;	Initializes the maximum profit to zero.
for (int i = 1; i < n; i++)	Traverses the remaining days from left to right.
prices[i] - minPrice	Calculates today's profit using the lowest earlier buying price.
if (currentProfit > maxProfit)	Checks whether today's transaction produces a better profit.
maxProfit = currentProfit;	Updates the maximum profit when a better opportunity is found.

Code	Explanation
if (prices[i] < minPrice)	Checks whether today's price is the new minimum.
minPrice = prices[i];	Updates the minimum price for future selling days.
return maxProfit;	Returns the maximum profit found.

Why do we update maxProfit before minPrice?

This is an important detail!

Consider:

prices = {7, 1, 5, 3, 6, 4}

When the algorithm reaches price 1, it first calculates the profit using the previous minimum price, 7.

currentProfit = 1 - 7 = -6

Then it updates minPrice from 7 to 1. This ensures that the current day's price is not used as both the buying price and selling price in the same transaction. The new minimum is available for future days.

6. Complete Dry Run

Input: prices = {7, 1, 5, 3, 6, 4}

Initial values

Variable	Value	Meaning
n	6	Total number of days
minPrice	7	Initial minimum price
maxProfit	0	Initial maximum profit

Step-by-step execution

The algorithm starts from index 1 because index 0 has already initialized minPrice.

Index	Today's Price	Minimum Before Update	Current Profit	Maximum Profit After Check	Minimum After Update
0	7	—	—	0	7
1	1	7	1 - 7 = -6	0	1
2	5	1	5 - 1 = 4	4	1
3	3	1	3 - 1 = 2	4	1
4	6	1	6 - 1 = 5	5	1
5	4	1	4 - 1 = 3	5	1

Iteration 1 — Price 1

Before this iteration:

- minPrice = 7
- maxProfit = 0

Calculate:

currentProfit = 1 - 7 = -6

Since -6 is not greater than 0, maxProfit remains 0. Now, 1 is less than the previous minimum of 7, so minPrice = 1. The algorithm has found a cheaper buying opportunity.

Iteration 2 — Price 5

Before this iteration:

- minPrice = 1
- maxProfit = 0

Calculate:

currentProfit = 5 - 1 = 4

Since 4 is greater than 0: maxProfit = 4. The minimum price remains 1 because 5 is not lower than 1.

Iteration 3 — Price 3

Before this iteration:

- minPrice = 1
- maxProfit = 4

Calculate:

currentProfit = 3 - 1 = 2

Since 2 is less than 4, the maximum profit remains 4. The minimum price also remains 1.

Iteration 4 — Price 6

Before this iteration:

- minPrice = 1
- maxProfit = 4

Calculate:

currentProfit = 6 - 1 = 5

Since 5 is greater than 4: maxProfit = 5. The minimum price remains 1. This is the best transaction found so far: buy at 1, sell at 6.

Iteration 5 — Price 4

Before this iteration:

- minPrice = 1
- maxProfit = 5

Calculate:

currentProfit = 4 - 1 = 3

Since 3 is less than 5, the maximum profit remains 5. The minimum price remains 1.

Final result

Maximum Profit: 5

The best transaction is to buy at price 1 and sell later at price 6.

7. Complexity Analysis

Time Complexity — O(n)

The algorithm traverses the array exactly once, starting from index 1. For each element, it performs a fixed number of operations:

1. Calculate the current profit.
2. Compare and potentially update maxProfit.
3. Compare and potentially update minPrice.

Each operation takes constant time, O(1). For an array of n elements, the total work grows linearly with the number of elements.

Therefore, **time complexity = O(n)**.

Auxiliary Space Complexity — O(1)

The algorithm uses only a fixed number of variables: n, minPrice, maxProfit, i, and currentProfit.

It does not create a prefix minimum array, suffix maximum array, or any other data structure that grows with the input.

Therefore, **auxiliary space = O(1)**.

8. Comparison With the Prefix Minimum Array

Property	Prefix Minimum Array	Greedy: Track Minimum
Main idea	Store the minimum price up to each index	Maintain the minimum price in one variable
Time complexity	O(n)	O(n)
Auxiliary space	O(n)	O(1)
Extra array	Required	Not required
Efficiency	Linear time	Linear time with constant extra space

The greedy approach preserves the linear-time logic while eliminating the extra array.

9. What I Learned

- I do not need to store every earlier price or examine every possible pair.
- For each selling day, the lowest earlier buying price gives the maximum possible profit for that day.
- minPrice stores the lowest price encountered so far.
- maxProfit stores the best profit found so far.
- Updating the profit before the minimum price preserves the buying-before-selling rule.
- The algorithm runs in O(n) time and uses O(1) auxiliary space.

Final takeaway: This is the optimal approach I should prioritize for interviews because it is efficient, easy to explain, and uses constant auxiliary space.

A useful next concept to explore is the connection to Kadane's algorithm, which solves this same problem by treating consecutive price changes as an array of gains and losses.

Approach 6 — Kadane's Algorithm Using Price Differences

1. Initial Intuition — A Different Way to Think About Profit

So far, we have calculated profit directly:

Profit = Selling Price − Buying Price

In this approach, instead of directly comparing buying and selling prices, we calculate the change in price between every pair of consecutive days.

Consider the input:

prices = [7, 1, 5, 3, 6, 4]

The consecutive price differences are:

From day	To day	Calculation	Difference
0	1	1 - 7	-6
1	2	5 - 1	+4
2	3	3 - 5	-2
3	4	6 - 3	+3
4	5	4 - 6	-2

Therefore:

diff = [-6, 4, -2, 3, -2]

Each difference tells us how much the stock price increased or decreased between consecutive days.

- A positive difference means the price increased.
- A negative difference means the price decreased.
- A zero difference means the price stayed the same.

2. Why Do Price Differences Help Us Calculate Profit?

Suppose I buy the stock at price 1 and sell it later at price 6. The prices between those days are:

1 → 5 → 3 → 6

The consecutive changes are:

- 5 - 1 = +4
- 3 - 5 = -2
- 6 - 3 = +3

Adding these differences:

4 + (-2) + 3 = 5

Directly calculating the profit gives the same result:

6 − 1 = 5

Why? Because the intermediate prices cancel when the differences are added:

(5 − 1) + (3 − 5) + (6 − 3) = 6 − 1

The result is the total change from the buying price to the selling price.

My key observation: The profit from buying on one day and selling on a later day equals the sum of the consecutive price differences between those days.

3. Connecting This Problem to Kadane's Algorithm

Kadane's algorithm finds the maximum-sum contiguous subarray in an array.

We have transformed the stock prices into a difference array:

diff = [-6, 4, -2, 3, -2]

Now our task is to find the contiguous section with the maximum sum.

For example, the maximum-sum contiguous section is:

4 + (-2) + 3 = 5

Maximum profit = 5

The contiguous section [4, -2, 3] represents the price changes from buying at 1 to selling at 6.

We can therefore solve the stock problem by applying Kadane's algorithm to the consecutive price differences.

4. Understanding the Two Variables

We use two variables:

- currentSum — the sum of the current contiguous section of price differences.
- maxSum — the maximum profit found so far.

Both start at 0, because the problem allows us to return zero if no profitable transaction exists.

A. Understanding currentSum

```
currentSum = max(0, currentSum + diff);
```

This line compares two possibilities:

1. Extend the current section by adding the new difference.
2. Reset the current sum to 0 if the resulting sum becomes negative.

Why reset it? A negative accumulated sum cannot improve a future profit compared with starting a new section after that point. Keeping it would only reduce the sum of any future section we extend.

For example, if currentSum = 0 and diff = -6:

currentSum = max(0, 0 + (-6)) = 0

We discard that negative running sum and continue looking for a profitable section.

B. Understanding maxSum

```
maxSum = max(maxSum, currentSum);
```

This keeps track of the best sum encountered so far.

Suppose maxSum = 4 and the new currentSum = 2. The maximum remains 4. But if currentSum = 5, the maximum becomes 5.

Remember: currentSum tracks the current section, while maxSum remembers the best result across all sections.

5. Complete C++ Code

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

// Problem: 121. Best Time to Buy and Sell Stock
// Approach 6: Kadane's Algorithm using Price Differences
// Time Complexity: O(n)
// Auxiliary Space: O(1)

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();

        int currentSum = 0;
        int maxSum = 0;

        // Calculate consecutive price differences
        for (int i = 1; i < n; i++) {
            int diff = prices[i] - prices[i - 1];

            // Extend the current section or reset its sum
            currentSum = max(0, currentSum + diff);

            // Store the maximum profit found so far
            maxSum = max(maxSum, currentSum);
        }

        return maxSum;
    }
};

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    int profit = solution.maxProfit(prices);

    cout << "Maximum Profit: " << profit << endl;

    return 0;
}
```

6. Detailed Dry Run

Input: prices = [7, 1, 5, 3, 6, 4]

The consecutive differences are:

diff = [-6, 4, -2, 3, -2]

Initial values:

- currentSum = 0
- maxSum = 0

Step-by-step execution

i	prices[i] - prices[i-1]	Calculation for currentSum	currentSum after update	maxSum after update
1	-6	max(0, 0 - 6)	0	0
2	+4	max(0, 0 + 4)	4	4
3	-2	max(0, 4 - 2)	2	4
4	+3	max(0, 2 + 3)	5	5
5	-2	max(0, 5 - 2)	3	5

Understanding each iteration

Iteration 1 — Difference = -6

The price falls from 7 to 1.

currentSum = max(0, -6) = 0

The negative sum is discarded. No profit has been found yet, so maxSum stays 0.

Iteration 2 — Difference = +4

The price rises from 1 to 5.

currentSum = max(0, 0 + 4) = 4

The best profit so far becomes 4.

Iteration 3 — Difference = -2

The price falls from 5 to 3.

currentSum = max(0, 4 - 2) = 2

We keep the current section because its sum is still positive. The best profit remains 4.

Iteration 4 — Difference = +3

The price rises from 3 to 6.

currentSum = max(0, 2 + 3) = 5

The best profit becomes 5.

Iteration 5 — Difference = -2

The price falls from 6 to 4.

currentSum = max(0, 5 - 2) = 3

The current sum falls to 3, but maxSum remains 5.

Final result: Maximum Profit: 5

The maximum-sum section is [4, -2, 3], corresponding to buying at 1 and selling at 6.

7. Complexity Analysis

Time complexity: O(n)

The loop starts at index 1 and processes each consecutive price difference exactly once. Each iteration performs a fixed number of arithmetic and comparison operations.

For n prices, there are n - 1 differences, so the time complexity is **O(n)**.

Auxiliary space: O(1)

We only maintain a fixed number of variables: n, currentSum, maxSum, and diff. We do not create a separate difference array in the implementation.

Although we describe the difference array conceptually, the code calculates each difference as needed. Therefore, the auxiliary space complexity is **O(1)**.

8. What I Learned

- Consecutive price differences can represent the total profit between two days.
- The stock-profit problem can be transformed into a maximum-sum contiguous subarray problem.
- Kadane's algorithm uses currentSum to track a current section and maxSum to preserve the best result.
- Resetting a negative running sum prevents it from reducing the profit of a future section.
- This implementation achieves O(n) time and O(1) auxiliary space.

Complete Comparison of All Six Approaches

No.	Approach	Main Idea	Time Complexity	Auxiliary Space
1	Brute Force	Check every valid buying and selling pair to find the maximum stock profit.	$O(n^2)$	O(1)
2	max_element()	For each buying day, find the maximum price among all future days.	$O(n^2)$	O(1)
3	Suffix Maximum Array	Precompute the maximum future price for each day and use it to calculate profit.	O(n)	O(n)
4	Prefix Minimum Array	Precompute the minimum price from the beginning up to each day to find the best selling profit.	O(n)	O(n)
5	Greedy — Minimum Price	Track the minimum price seen so far and update the maximum profit at each day.	O(n)	O(1)
6	Kadane's Algorithm	Find the maximum contiguous sum of price differences to determine the maximum profit.	O(n)	O(1)

Final Learning Summary

The approaches progress from checking every valid transaction to using precomputed information and then maintaining only the information needed at each step.

- **Approach 1 — Brute Force:** Checks every valid pair; $O(n^2)$ time and O(1) auxiliary space.
- **Approach 2 — max_element():** Finds the best future selling price repeatedly; $O(n^2)$ time and O(1) auxiliary space.
- **Approach 3 — Suffix Maximum Array:** Precomputes future maximum prices; O(n) time and O(n) auxiliary space.
- **Approach 4 — Prefix Minimum Array:** Precomputes earlier minimum prices; O(n) time and O(n) auxiliary space.
- **Approach 5 — Greedy Minimum Price:** Tracks the lowest price so far and the best profit; O(n) time and O(1) auxiliary space.
- **Approach 6 — Kadane's Algorithm:** Treats consecutive price changes as gains and losses; O(n) time and O(1) auxiliary space.

Interview priority: Focus first on Approach 5, Greedy — Track Minimum Price. It is the clearest optimal solution to explain and implement. Approach 6 is a useful conceptual connection to maximum subarray sum.

Details of Kadane's Algorithm

LeetCode 121 — Best Time to Buy and Sell Stock | Approach 6: Using Price Differences

Purpose of these notes: Record the reasoning behind Kadane's Algorithm, the doubts that came up while learning it, and how the final understanding connects price differences to maximum stock profit.

1. My Initial Understanding

Until this approach, I calculated profit directly using **Profit = Selling Price** – **Buying Price**. For example, buying at 1 and selling later at 6 gives 6 – 1 = 5.

Kadane's approach looks at the change between each pair of consecutive days instead of directly checking every buy/sell pair.

```
prices = [7, 1, 5, 3, 6, 4]
diff   = [-6, +4, -2, +3, -2]
```

From price	To price	Calculation	Difference
7	1	1 – 7	–6
1	5	5 – 1	+4
5	3	3 – 5	–2
3	6	6 – 3	+3
6	4	4 – 6	–2

A positive difference means the price rose between those days; a negative difference means it fell.

2. Main Doubt: Why Does Adding Differences Give Profit?

Suppose the buying price is 1 and the later selling price is 6. The prices in between are 1 → 5 → 3 → 6, so the differences are +4, –2, and +3.

```
+4 + (-2) + 3 = 5
6 - 1 = 5
```

These results match because the intermediate prices cancel when consecutive differences are added:

```
(5 - 1) + (3 - 5) + (6 - 3) = 6 - 1
```

So the sum of consecutive price movements gives the overall movement from the starting price to the ending price. In this example, +4 – 2 + 3 = 5 represents buying at 1 and selling at 6.

3. What Does currentSum Store?

currentSum is the accumulated sum of the current continuous section of differences being considered. It is not one permanent sum that must include every difference.

For example, after processing +4 and –2:

```
currentSum = +4 + (-2) = 2
```

The current section has an overall movement of +2. When the next difference +3 arrives, Kadane decides whether to extend this section or start a new one.

4. The Key Decision: Continue or Start Fresh?

At each difference, Kadane asks: **Should I extend the current section, or would starting fresh here give a better sum?**

When we reach +3, the existing currentSum is 2.

Choice	Section	Sum	Interpretation
Continue	[+4, –2, +3]	2 + 3 = 5	Price movement from 1 to 6
Start fresh	[+3]	3	Price movement from 3 to 6

> Continuing gives 5, which is better than 3, so the current section is extended. A negative difference does not automatically mean the section should be discarded; +4 – 2 is still +2.

5. Understanding currentSum = max(0, currentSum + diff)

Part	Meaning
currentSum + diff	Extend the current section by adding the new difference.
0	Do not carry a negative accumulated sum forward; reset the running sum to zero.
max(...)	Keep the greater of zero and the extended sum.

Starting fresh is handled naturally. If currentSum is 0 and diff is +4, then max(0, 0 + 4) = 4. If currentSum is 0 and diff is –6, then max(0, 0 – 6) = 0.

A negative difference does not itself trigger a reset. For example, currentSum = 4 and diff = –2 gives max(0, 4 – 2) = 2, so the section continues.

Why discard a negative running sum? A negative sum hurts any later section we extend it into. Starting from zero instead avoids carrying that loss forward. In this stock problem, zero is also a valid answer because making no transaction gives profit 0.

6. Difference Between currentSum and maxSum

Variable	Responsibility
currentSum	The accumulated sum of the current continuous section.
maxSum	The highest sum / profit found so far, preserved even if currentSum later decreases.

When currentSum reaches 5, maxSum is updated to 5. If the next difference is –2, currentSum drops to 3, but maxSum remains 5 because that better result has already been found.

7. Complete Dry Run

Input: prices = [7, 1, 5, 3, 6, 4] | Differences: [–6, +4, –2, +3, –2]

Difference	Calculation	currentSum after update	maxSum after update
–6	max(0, 0 – 6)	0	0
+4	max(0, 0 + 4)	4	4
–2	max(0, 4 – 2)	2	4
+3	max(0, 2 + 3)	5	5
–2	max(0, 5 – 2)	3	5

Step 1 — –6: The price drops from 7 to 1. The running sum would be negative, so it resets to 0; maxSum stays 0.

Step 2 — +4: The price rises from 1 to 5. currentSum becomes 4 and maxSum becomes 4.

Step 3 — −2: The price falls from 5 to 3. The running sum becomes 2, which is still positive, so we keep the section. maxSum stays 4.

Step 4 — +3: The price rises from 3 to 6. currentSum becomes 5 and maxSum improves to 5. The section +4 – 2 + 3 represents buying at 1 and selling at 6.

Step 5 — −2: The price falls from 6 to 4. currentSum decreases to 3, but maxSum remains 5.

Final answer: maxSum = 5.

8. Complete C++ Implementation

```
// Problem: 121. Best Time to Buy and Sell Stock
// Approach 6: Kadane's Algorithm Using Price Differences
// Time Complexity: O(n)
// Auxiliary Space: O(1)

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int n = prices.size();

        int currentSum = 0;
        int maxSum = 0;

        // Process each consecutive price difference
        for (int i = 1; i < n; i++) {
            int diff = prices[i] - prices[i - 1];

            // Extend the current section or discard
            // a negative accumulated sum
            currentSum = max(0, currentSum + diff);

            // Preserve the best profit found so far
            maxSum = max(maxSum, currentSum);
        }

        return maxSum;
    }
};

int main() {
    vector<int> prices = {7, 1, 5, 3, 6, 4};

    Solution solution;
    cout << "Maximum Profit: "
         << solution.maxProfit(prices) << endl;

    return 0;
}
```

Output:

```
Maximum Profit: 5
```

9. Complexity Analysis

Time Complexity: O(n). The loop processes each consecutive difference once. An array with n prices has n – 1 differences, and every iteration performs constant-time operations.

Auxiliary Space: O(1). The implementation calculates each difference when needed and stores only a fixed number of variables. It does not allocate a separate difference array.

10. My Final Understanding

• Instead of checking every buying/selling pair, calculate consecutive price differences.

• Adding consecutive differences gives the same result as ending price minus starting price because intermediate prices cancel.

• currentSum stores the accumulated sum of the current continuous section.

• At each difference, decide whether to continue the section or discard a negative accumulated sum.

• A negative difference alone does not mean reset. For example, 4 + (–2) = 2, so keep the section.

• maxSum remembers the best profit found so far, even if currentSum later decreases.

• The section [+4, –2, +3] represents the movement from price 1 to price 6, producing profit 5.

• The algorithm runs in O(n) time and uses O(1) auxiliary space.

Core takeaway: currentSum tracks the current section's accumulated price movement, while maxSum remembers the highest profit found so far. We can reset currentSum without losing the best result already discovered.